

Oriented Bounding Box Object Detection

Oriented Bounding Box Object Detection

- 1. Model Introduction
- 2. Start
 - 2.1. Enter docker
 - 2.2. Directed bounding box object detection: image
 - Effect preview
 - 2.3, directional border object detection: video
 - Effect preview
 - 2.4, directional border object detection: real-time detection
 - Effect preview
- References

Use Python to demonstrate the effect of Ultralytics: Oriented Bounding Boxes Object Detection in image, video, and real-time detection.

1. Model Introduction

Oriented Bounding Box Object Detection, also known as Oriented Object Detection, goes a step further than standard object detection. It introduces an additional angle to more accurately locate objects in the image.

The output of the oriented object detector is a set of rotated bounding boxes that accurately surround objects in the image, as well as the category label and confidence score of each bounding box. Oriented bounding boxes are particularly useful when objects appear at different angles, such as in aerial images, where traditional axis-aligned bounding boxes may include unnecessary background.

Simply put, oriented object detection can accurately surround tilted objects with tilted boxes, thereby reducing unnecessary background areas and improving detection accuracy.

2. Start

2.1. Enter docker

Run YOLOv11's docker script

```
sh ~/yolov11_docker.sh
```

2.2. Directed bounding box object detection: image

Use yolo11n-obb.pt to predict images under the ultralytics project (not the images that come with ultralytics).

Enter the code folder:

```
cd /ultralytics/ultralytics/yahboom_demo
```

Run the code:

```
python3 05.obb_image.py
```

Effect preview

Yolo recognizes the output image location: /ultralytics/ultralytics/output/

1. View using jupyter lab

Open another terminal to enter the docker container and use jupyter lab to view the image

```
docker ps -a
```

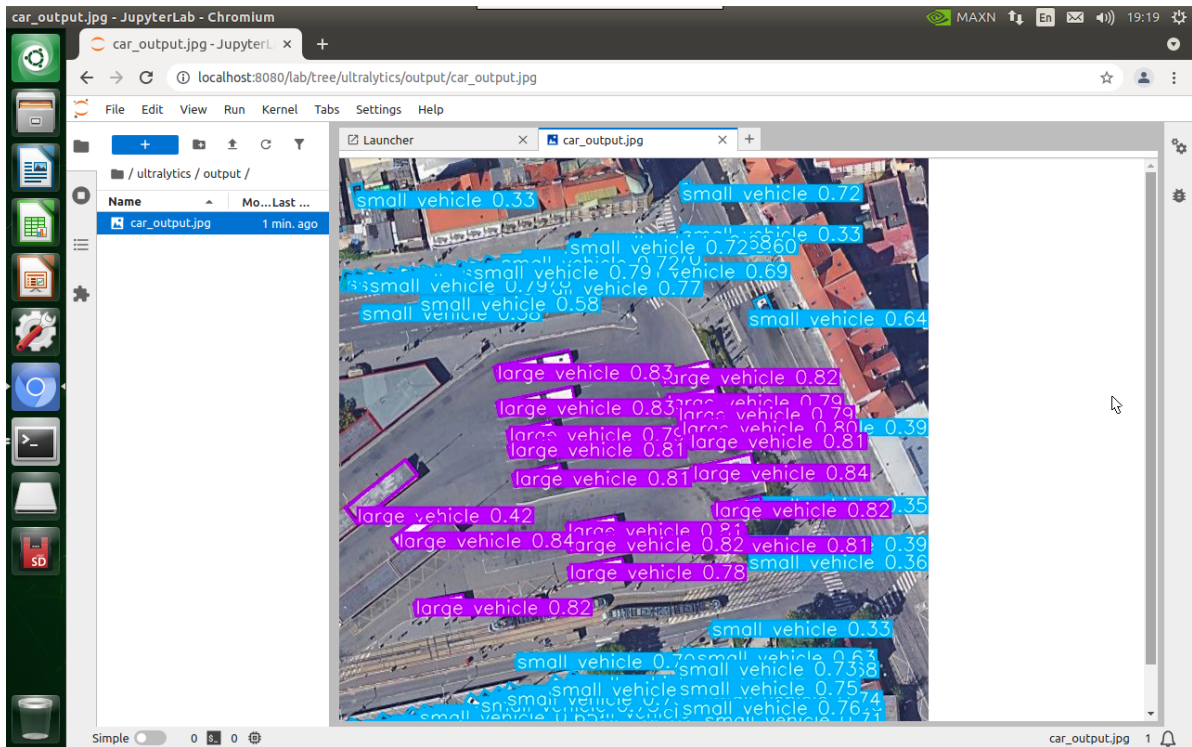
```
jetson@yahboom: ~  
jetson@yahboom:~$ docker ps -a  
CONTAINER ID   IMAGE                                COMMAND                  CREATED  
STATUS        PORTS          NAMES  
be79bf10e970   yahboomtechnology/ultralytics:1.0.2  "/bin/bash"            41 minutes ago  
Up 41 minutes 4520152747    trusting_tesla
```

```
docker exec -it be79bf10e970 /bin/bash #Container ID needs to be modified  
according to the actual one you find  
cd /ultralytics  
jupyter lab --allow-root
```

```
root@yahboom: /ultralytics  
root@yahboom:/ultralytics# jupyter lab --allow-root  
[I 2025-03-24 07:38:52.970 ServerApp] jupyter_lsp | extension was successfully l  
inked.  
[I 2025-03-24 07:38:52.985 ServerApp] jupyter_server_terminals | extension was s  
uccessfully linked.  
[I 2025-03-24 07:38:53.011 ServerApp] jupyterlab | extension was successfully li  
nked.  
[I 2025-03-24 07:38:53.776 ServerApp] notebook_shim | extension was successfully  
linked.  
[I 2025-03-24 07:38:53.865 ServerApp] notebook_shim | extension was successfully  
loaded.  
[I 2025-03-24 07:38:53.875 ServerApp] jupyter_lsp | extension was successfully l  
oaded.  
[I 2025-03-24 07:38:53.879 ServerApp] jupyter_server_terminals | extension was s  
uccessfully loaded.  
[I 2025-03-24 07:38:53.883 LabApp] JupyterLab extension loaded from /usr/local/l  
ib/python3.8/dist-packages/jupyterlab  
[I 2025-03-24 07:38:53.884 LabApp] JupyterLab application directory is /usr/loca  
l/share/jupyter/lab  
[I 2025-03-24 07:38:53.885 LabApp] Extension Manager is 'pypi'.  
[I 2025-03-24 07:38:54.000 ServerApp] jupyterlab | extension was successfully lo  
aded.  
[I 2025-03-24 07:38:54.001 ServerApp] Serving notebooks from local directory: /u  
ltralytics
```

Access directly through <http://localhost:8080/> in the system browser:

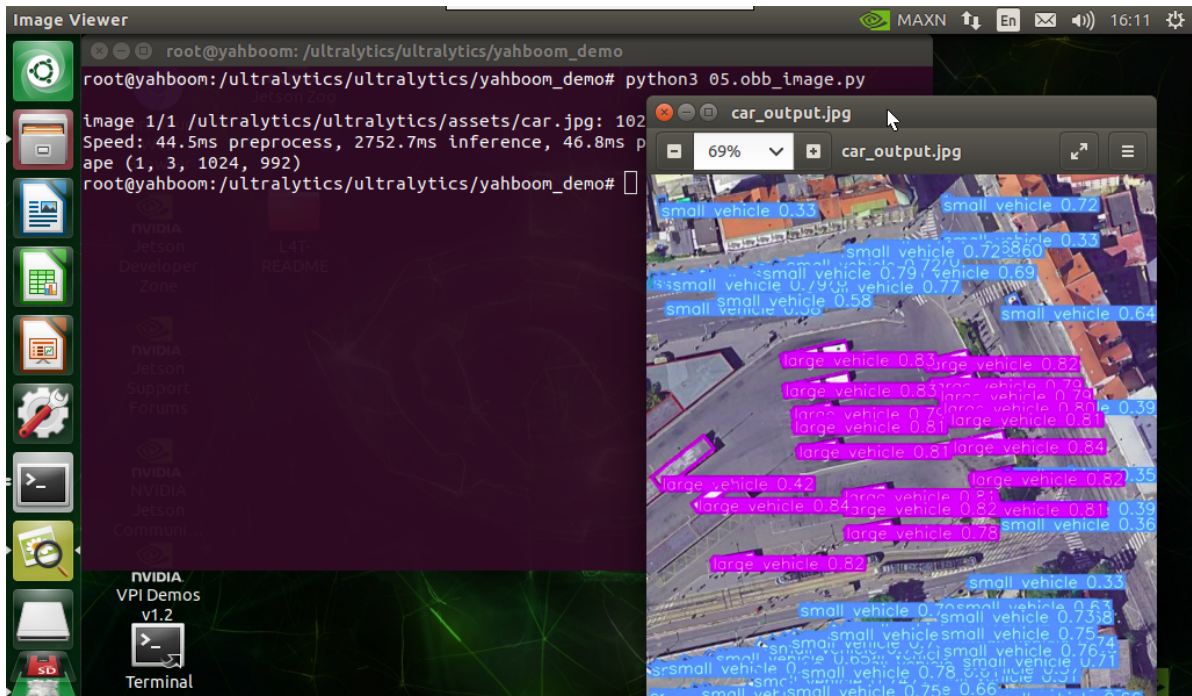
```
http://localhost:8080/
```



2. Copy the file to the host machine for viewing

Enter the following command in the host terminal

```
docker cp be79bf10e970:/ultralytics/ultralytics/output/
/home/jetson/ultralytics/ultralytics/ #Container ID needs to be modified
according to the actual information you find
```



Sample code:

```
from ultralytics import YOLO

# Load a model
model = YOLO("/ultralytics/ultralytics/yolo11n-obb.pt")

# Run batched inference on a list of images
```

```

results = model("/ultralytics/ultralytics/assets/car.jpg") # return a list of
Results objects

# Process results list
for result in results:
    # boxes = result.boxes # Boxes object for bounding box outputs
    # masks = result.masks # Masks object for segmentation masks outputs
    # keypoints = result.keypoints # Keypoints object for pose outputs
    # probs = result.probs # Probs object for classification outputs
    obb = result.obb # Oriented boxes object for OBB outputs
    result.show() # display to screen
    result.save(filename="/ultralytics/ultralytics/output/car_output.jpg") #
    save to disk

```

2.3, directional border object detection: video

Use yolo11n-obb.pt to predict the video under the ultralytics project (not the video that comes with ultralytics).

Enter the code folder:

```
cd /ultralytics/ultralytics/yahboom_demo
```

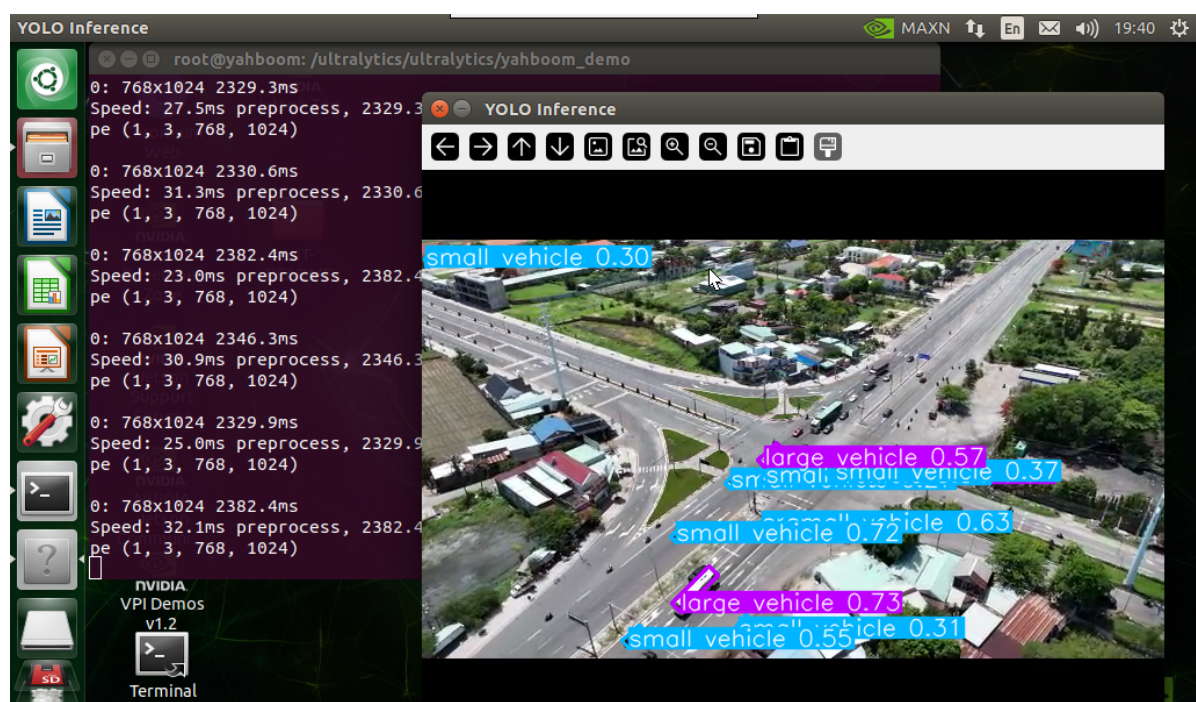
Run the code:

```
python3 05.obb_video.py
```

Effect preview

Yolo recognizes the output video location: /ultralytics/ultralytics/output/

The output video will be displayed in real time during the code running. If you want to view the video later, you can refer to the above **[2. Copy the file to the host machine for viewing]** tutorial operation.



Sample code:


```

import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/ultralytics/ultralytics/yolo11n-obb.pt")

# Open the video file
video_path = "/ultralytics/ultralytics/videos/street.mp4"
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a Videowriter object to output the processed video
output_path = "/ultralytics/ultralytics/output/05.street_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

2.4, directional border object detection: real-time detection

Use yolo11n-obb.pt to predict the USB camera screen.

Enter the code folder:

```
cd /ultralytics/ultralytics/yahboom_demo
```

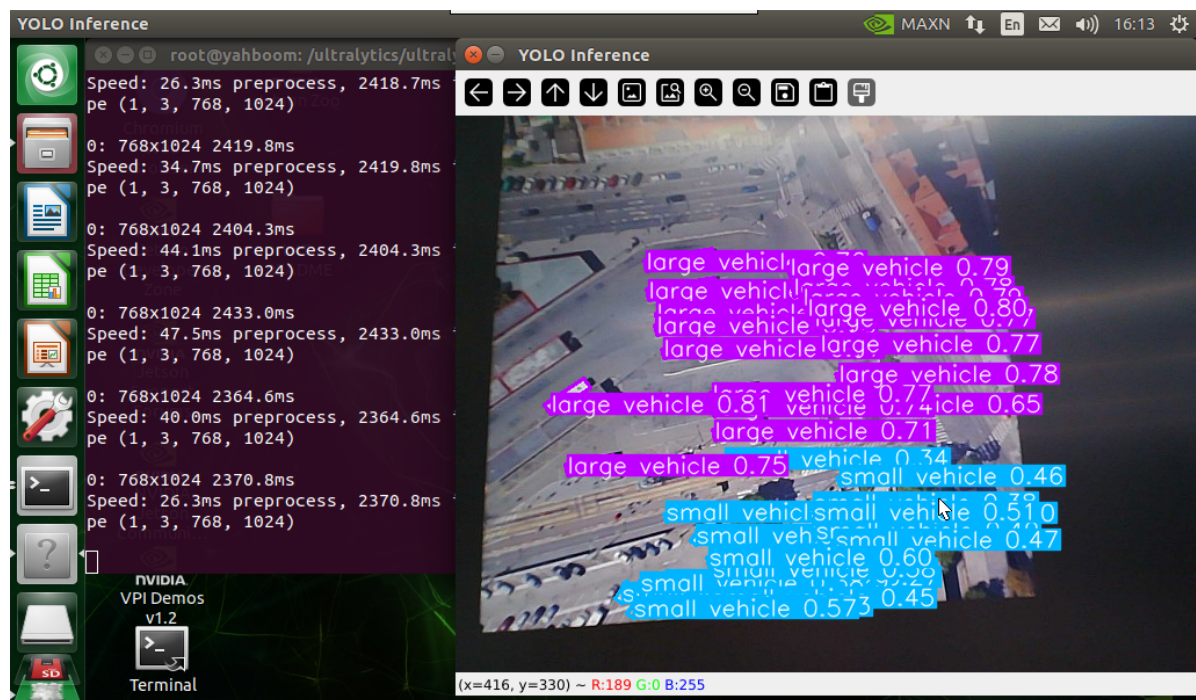
Run the code: click on the preview screen, press the q key to terminate the program!

```
python3 05.obb_camera_usb.py
```

Effect preview

Yolo recognizes the output video location: /ultralytics/ultralytics/output/

The camera screen will be displayed in real time during the code running. If you want to view the video later, you can refer to the above **[2. Copy the file to the host machine for viewing]** tutorial operation.



Sample code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/ultralytics/ultralytics/yolo11n-obb.pt")

# Open the camera
cap = cv2.VideoCapture(0)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a Videowriter object to output the processed video
output_path = "/ultralytics/ultralytics/output/05.obb_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))
```

```
# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()
```

References

<https://docs.ultralytics.com/tasks/obb/>